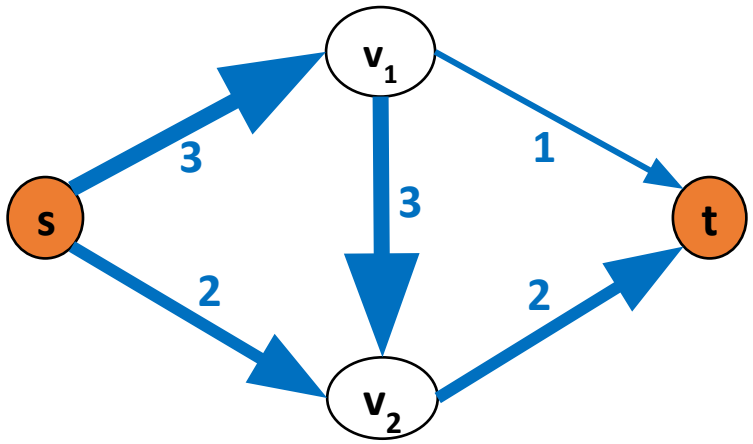
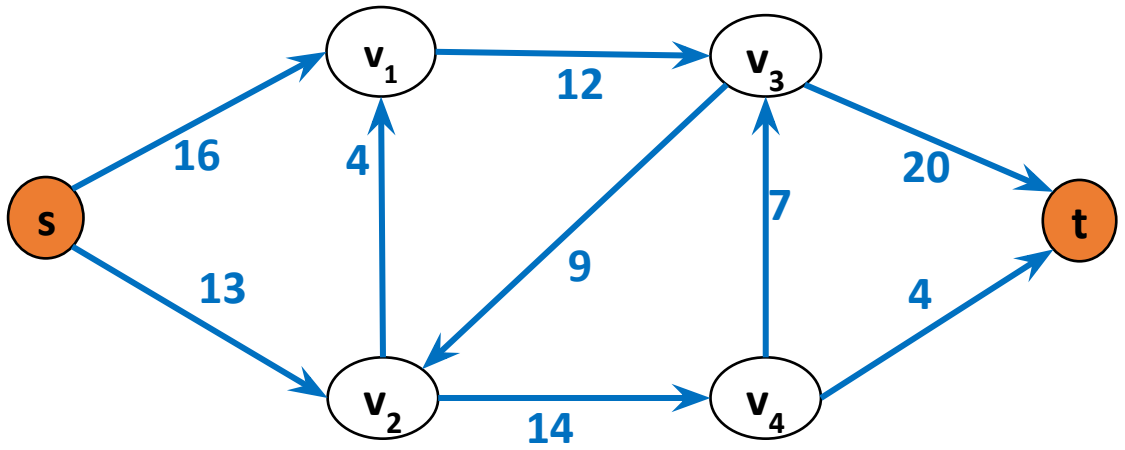
Flow Network



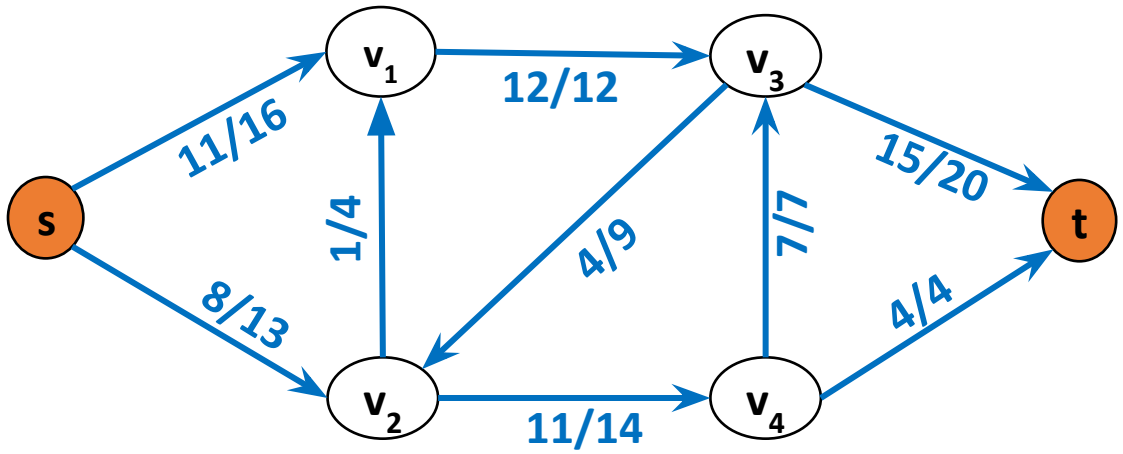
S. S. Satapathy

Flow network

- A **flow network G** = (V, E) is a **directed graph** in which **each edge** (u, v) ∈ E has a **nonnegative capacity** c(u, v) >= 0. The flow network contains two distinguished vertices: a **source s** and a **sink t**. A **flow** in G is a real-valued function *f*: V x V → R that satisfies following two properties:
  - **Capacity constraint: For all (u, v) ∈ E, 0 ≤ *f*(*u*,*v*) ≤ *c*(*u*,*v*)**. The **flow from one vertex to another** must be nonnegative and **must not exceed** the **given capacity**.
  - **Flow conservation: For all u ∈ V – {s, t}**, we require $\sum_{v \in V} f(v,u) = \sum_{v \in V} f(u,v)$. The **total flow into a vertex** other than the source or sink **must equal the total flow out of that vertex**, informally, "**flow in equals to flow out**".



A Flow network G

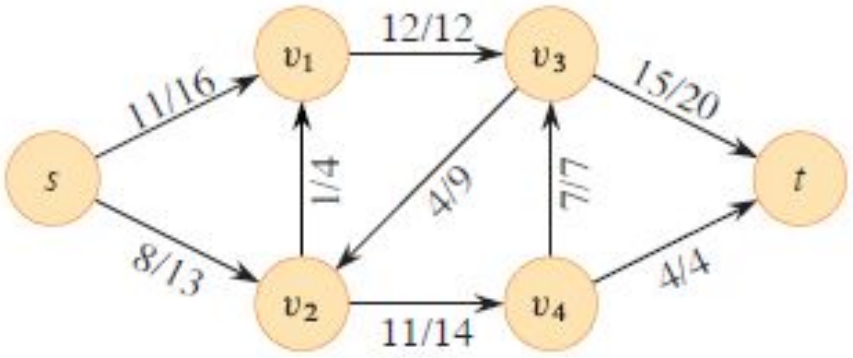


A flow *f* in G with value |*f*| = 19. Each edge (u,v) is labeled by *f*(u,v)/c(u,v)

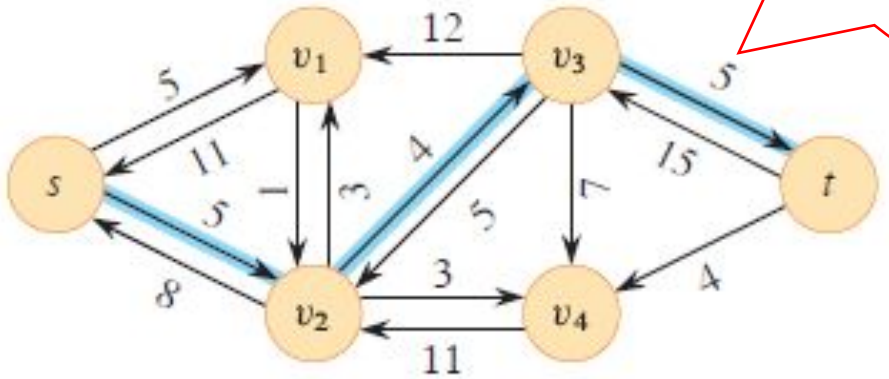
Ford-Fulkerson method for Max-flow Problem

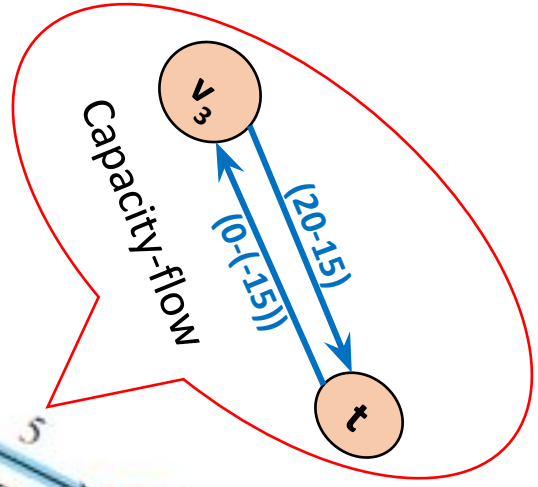
- In the **maximum-flow problem**, the **input is a flow network G** with **source s** and **sink t** , and the **goal is to find a flow of maximum value**.
- The **Ford-Fulkerson method** for **solving maximum-flow problem**, use **three concepts**: **1. residual networks**, **2. augmenting paths**, and **3. cuts**.
- **1.** in a **flow network G** = (**V**,**E**) with **source s** , **sink t** , and a **flow *f***, for a **pair of vertices** (**u**, **v**), the **residual capacity $c_f(u, v)$** is **defined by**

$$c_f(u,v) = \begin{cases} c(u,v) - f(u,v) & \text{if } (u,v) \in E\,, \\ f(v,u) & \text{if } (v,u) \in E\,, \\ 0 & \text{otherwise}\,. \end{cases}$$



The **flow network G** and flow *f*



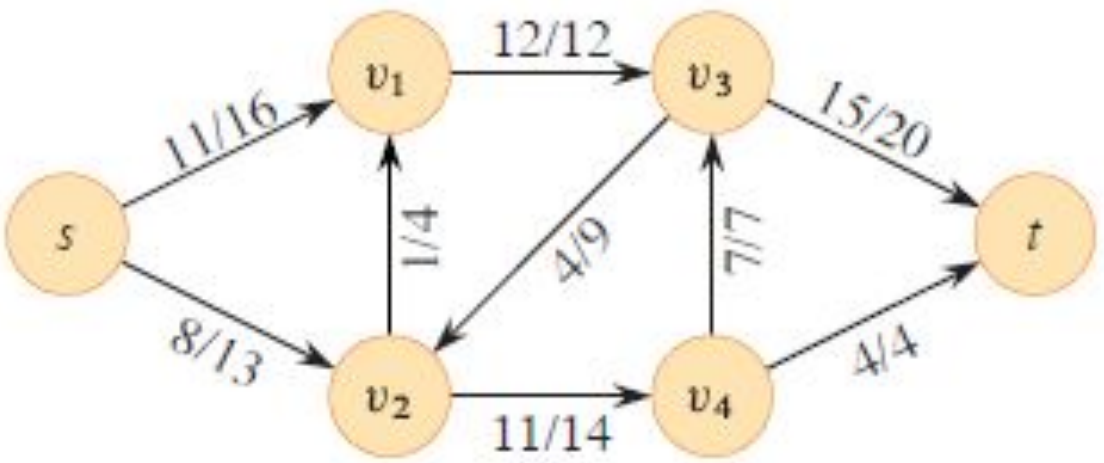


The **residual network G_f** with **augmenting path *p***, having **residual capacity** $c_f(p)=c_f(v_2,v_3)=$ **4** in **blue**.

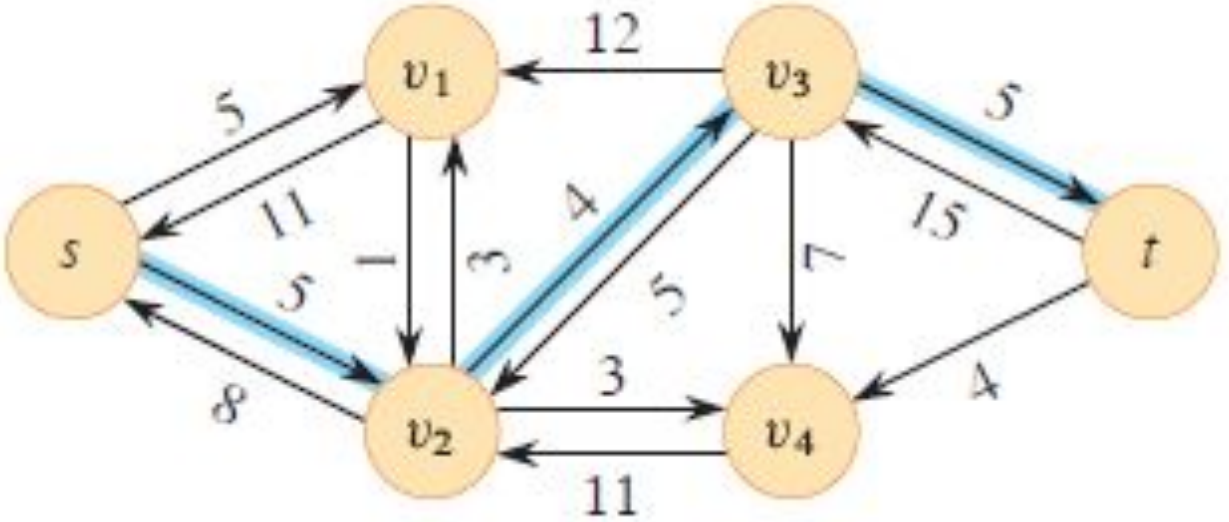
Ford-Fulkerson method for Max-flow Problem

- **2.** Given a flow network **G** = (V, E) and a flow ***f***, an ***augmenting path p*** is a simple path from **s** to **t** in the residual network G_f.
- The **maximum amount** by which we can **increase** the **flow** on **each edge in an augmenting path *p*** the ***residual capacity*** of ***p***, given by

$$c_f(p) = \min\{c_f(u,v) : (u,v) \text{ is in } p\}$$



The flow network G and flow *f*



The residual network G_f with augmenting path p, having residual capacity $c_f(p)=c_f(v_2,v_3)= 4$ in blue.

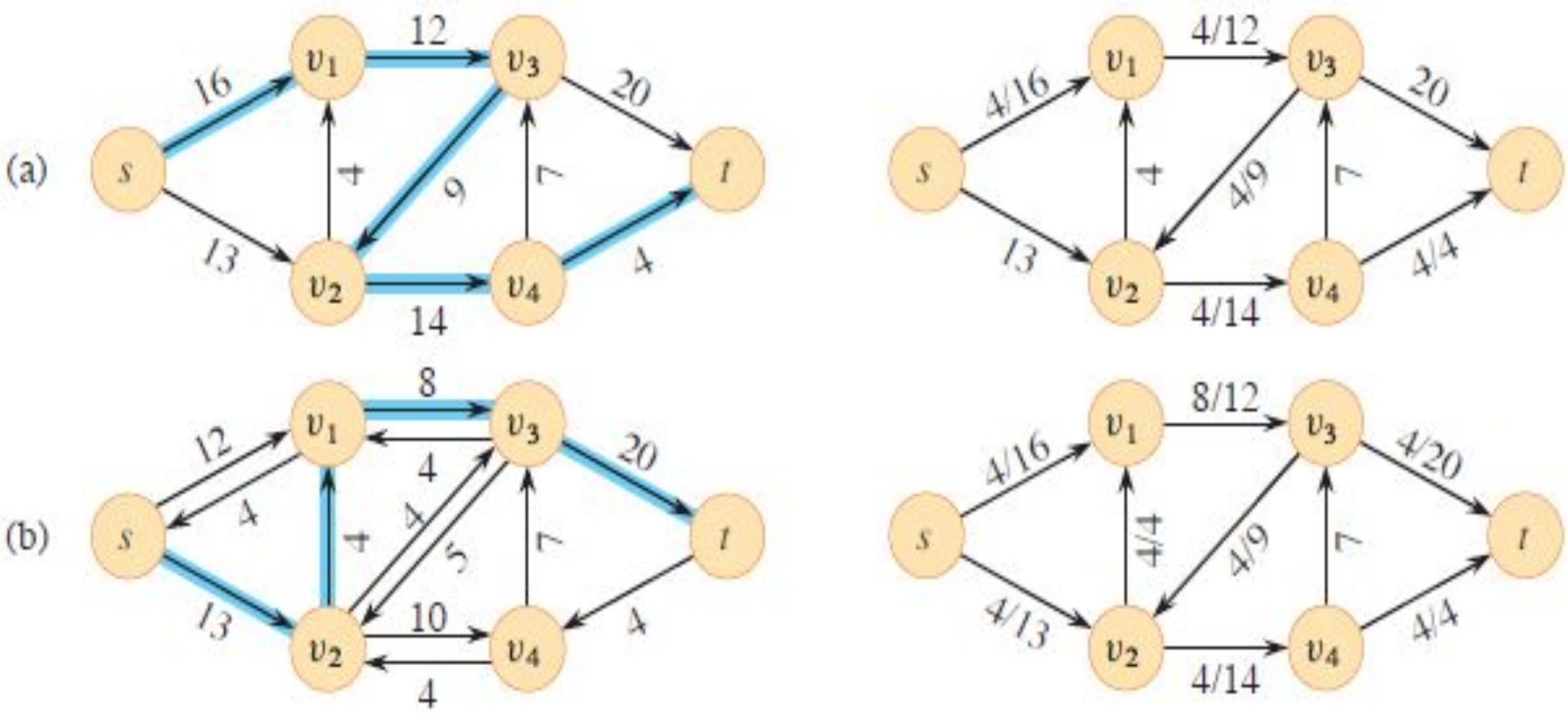
Ford-Fulkerson method for Max-flow Problem

- The **Ford-Fulkerson method** **repeatedly** **augments** **the flow along** **augmenting paths** **until** it has **found a** **maximum flow**. The **max-flow min-cut theorem**, **tells** that the **flow is maximum if and only if** its **residual network** **contains no** **augmenting path**.

FORD-FULKERSON (G, s, t)

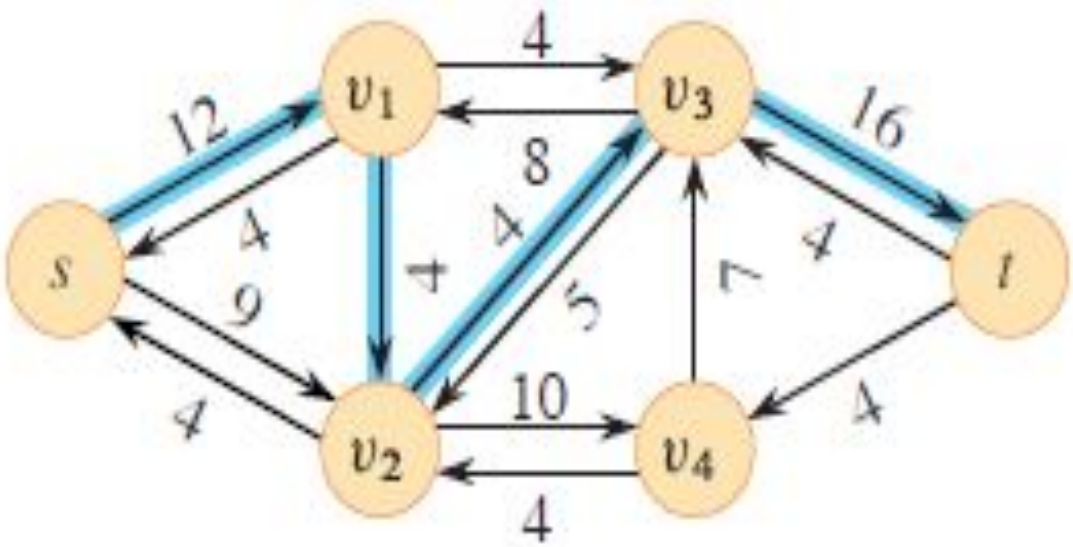
```
for each edge (u,v) in G
    f(u,v) = 0
while there exists a path p from s to t in the residual network G_f
    c_f(p) = min {c_f (u,v) : (u,v) is in p}
    for each edge (u,v) in p
        if (u,v) in G
            f(u,v) = f(u,v) + c_f(p)
        else f(v,u) = f(v,u) - c_f(p)
return f
```

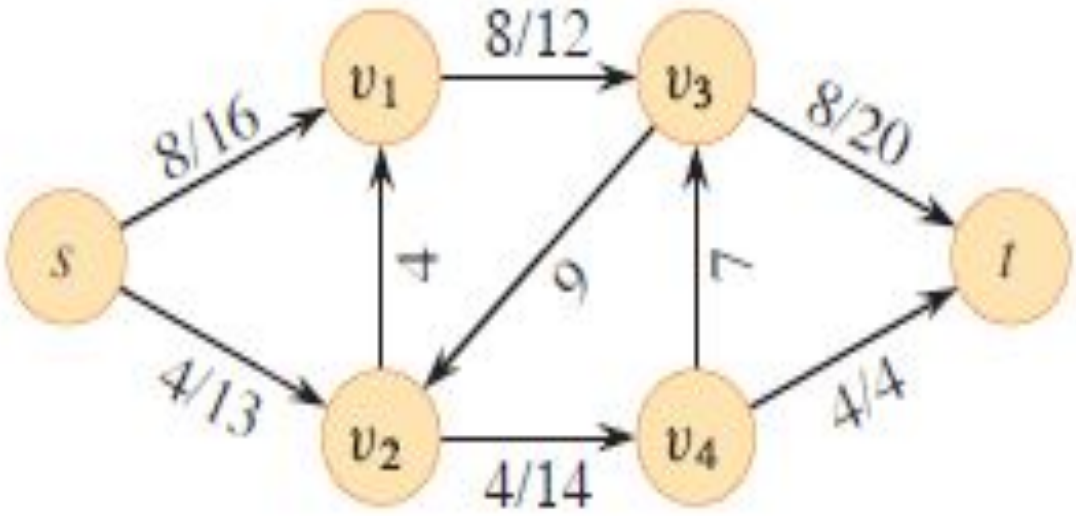
Execution of basic Ford-Fulkerson algorithm



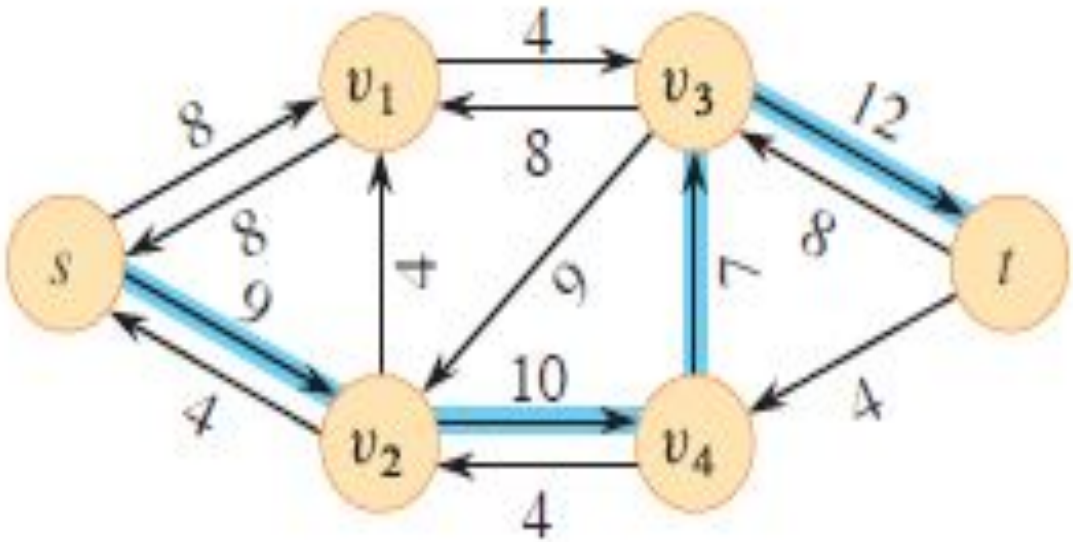
Execution of basic Ford-Fulkerson algorithm

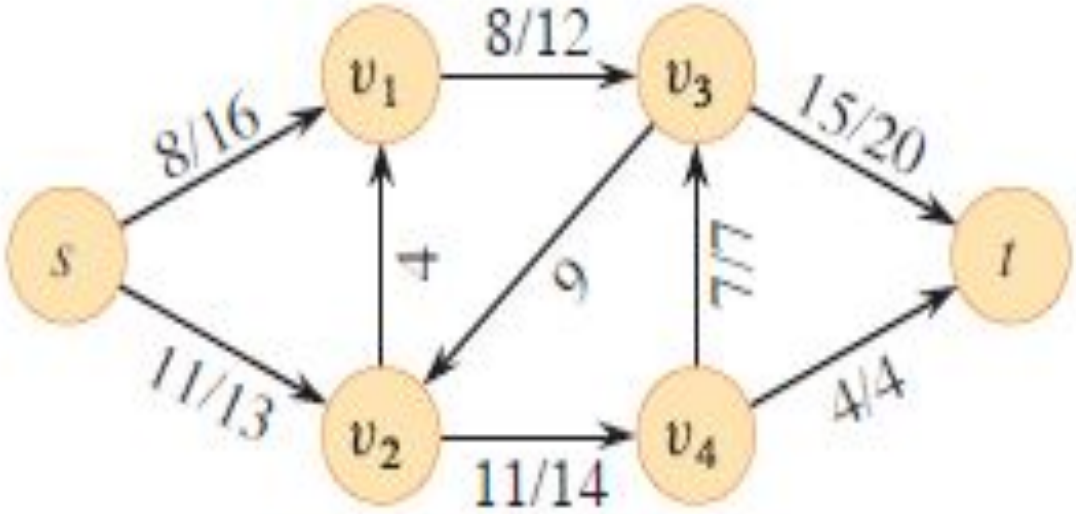
(c)





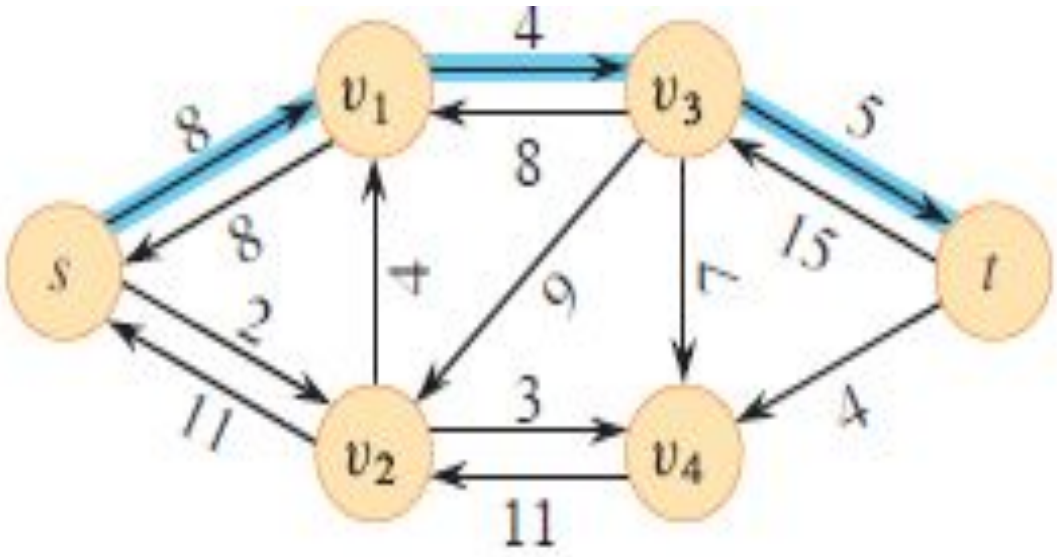
(d)

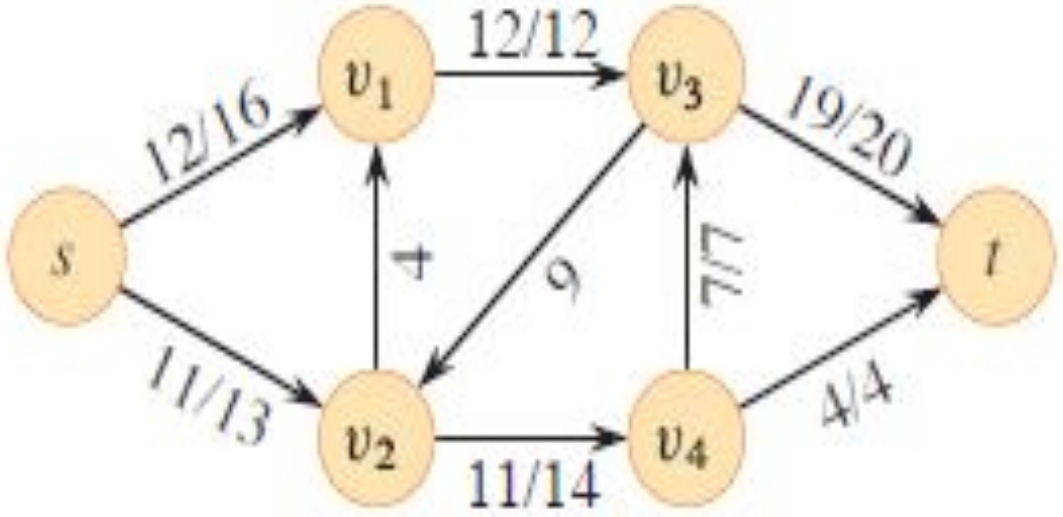




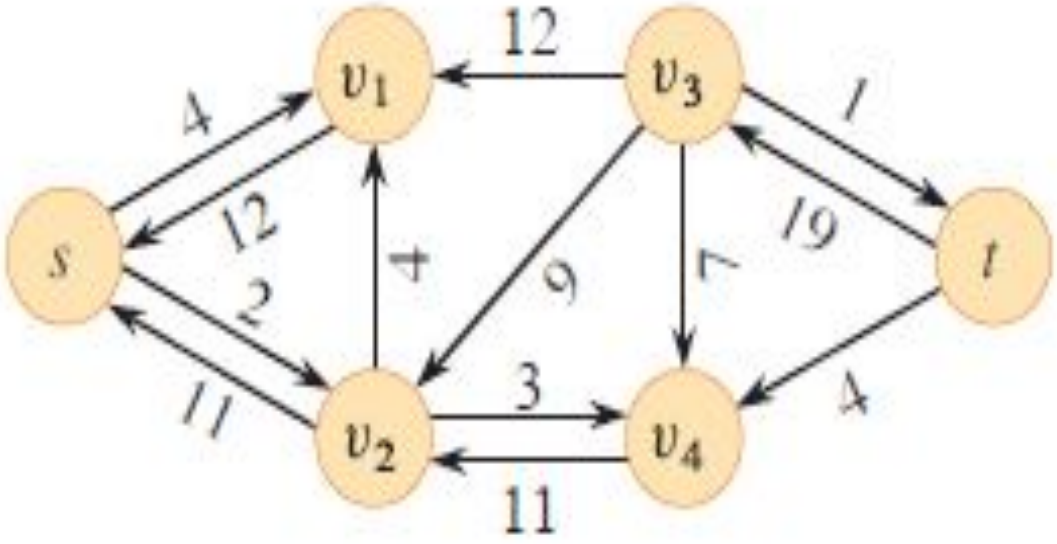
Execution of basic Ford-Fulkerson algorithm

(e)





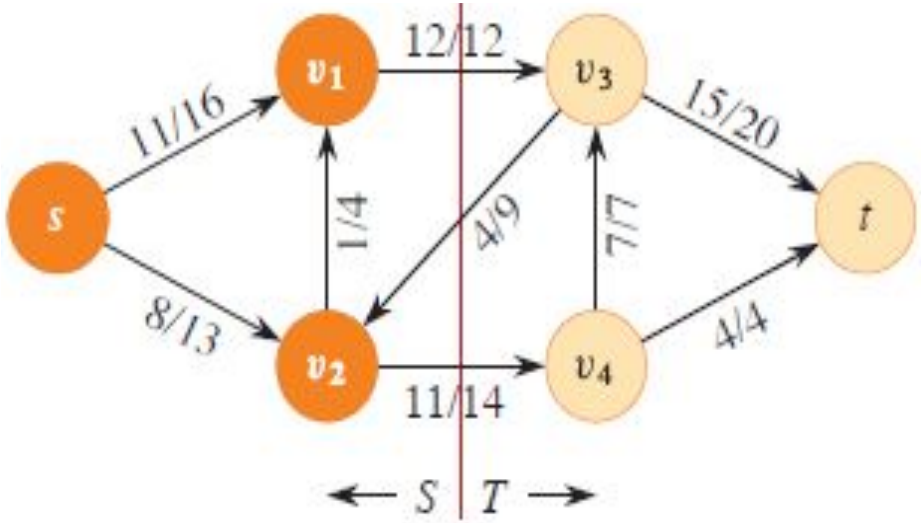
(f)



(a)–(e) Successive iterations of the **while** loop. The left side of each part shows the residual network G_f from line 3 with a blue augmenting path p. The right side of each part shows the new flow ***f*** that results from augmenting ***f*** by f_p. The residual network in (a) is the input flow network G. **(f)** The residual network at the last **while** loop test. It has no augmenting paths, and the flow ***f*** shown in (e) is therefore a maximum flow. The value of the **maximum flow found is 23**.

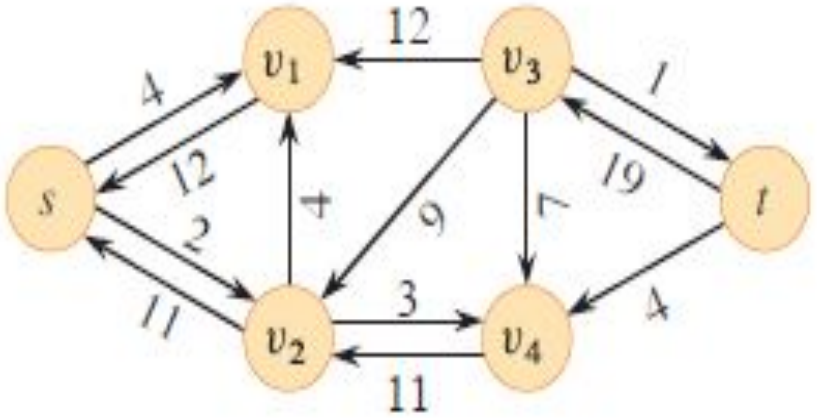
Ford-Fulkerson method for Max-flow Problem

- A **cut(S,T)** of **flow network G =(V,E)** is a **partition of V** into **S and T = V - S** such that **s ∈ S** and **t ∈ T** (This is a special type of cut in a directed graph rather than an undirected graph, and we insist that s ∈ S and t ∈ T.)
- If ***f*(u,v)** is a **flow**, then the ***net flow f* (S, T) across the cut (S,T)** is **defined** as $f(S,T) = \sum_{u \in S} \sum_{v \in T} f(u,v) - \sum_{u \in S} \sum_{v \in T} f(v,u)$
- The **capacity** of a **cut (S,T)** is **defined** as $c(S,T) = \sum_{u \in S} \sum_{v \in T} c(u,v)$
- A **minimum cut of a network** is a **cut** whose **capacity is minimum over all cuts of the network**.
- The **net flow across** the **cut** ({s,v_1,v_2}, {v_3,v_4,t}) in the flow network can be calculated as
  - **$f(v_1,v_3)+f(v_2,v_4)-f(v_3,v_2)=12+11-4=19$**
- **Capacity** of **this cut** is calculated as
  - **$c(v_1,v_3)+c(v_2,v_4)=12+14=26$**

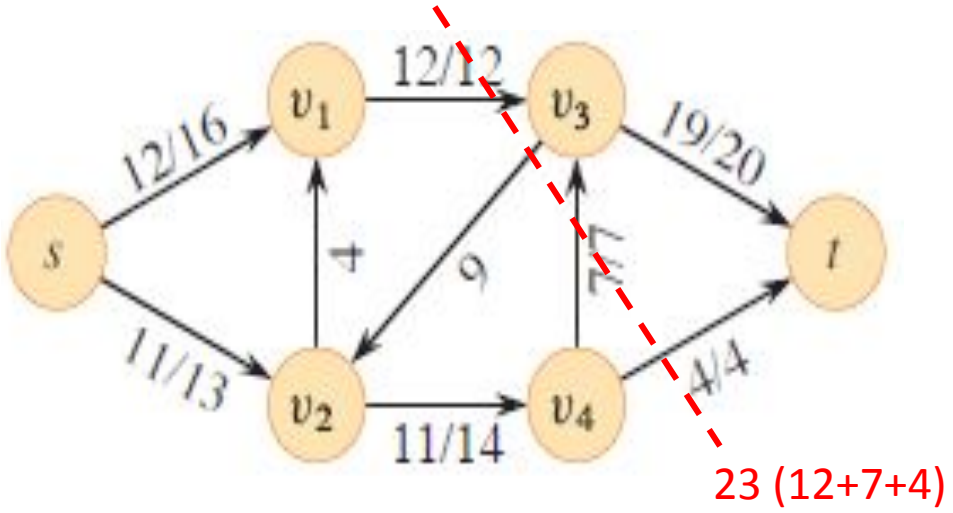


Ford-Fulkerson method for Max-flow Problem

- **Max-flow min-cut theorem:** (It says that the value of a maximum flow is in fact equal to the capacity of a minimum cut)
- If ***f*** is a flow in a flow network **G = (V , E)** with **source s** and **sink t**, then the **following conditions** are **equivalent**:
  - 1. ***f*** is a **maximum flow** in **G**.
  - 2. The **residual network G_f contains no augmenting paths**.
  - 3. **Net flow *f* = c(S, T) for some cut (S, T)** of G.



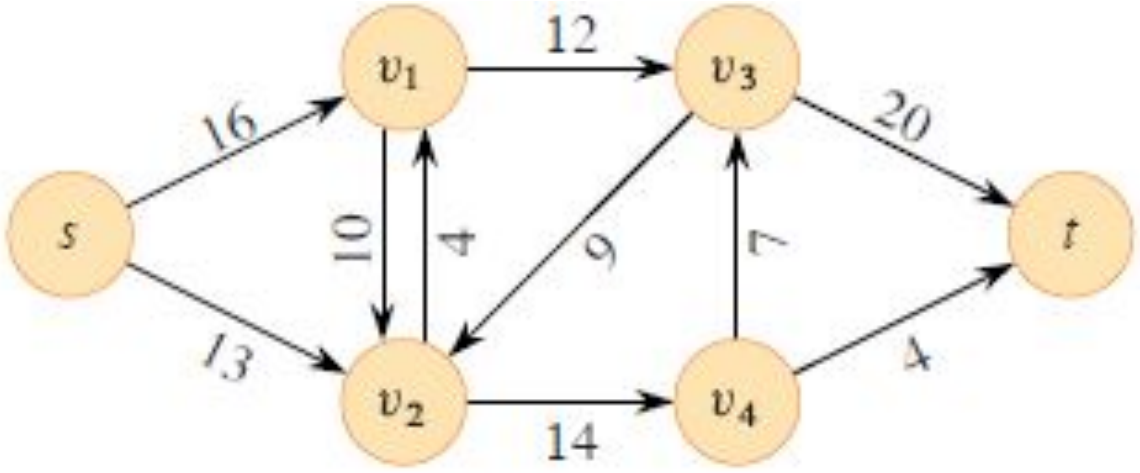
residual network contains no augmenting paths



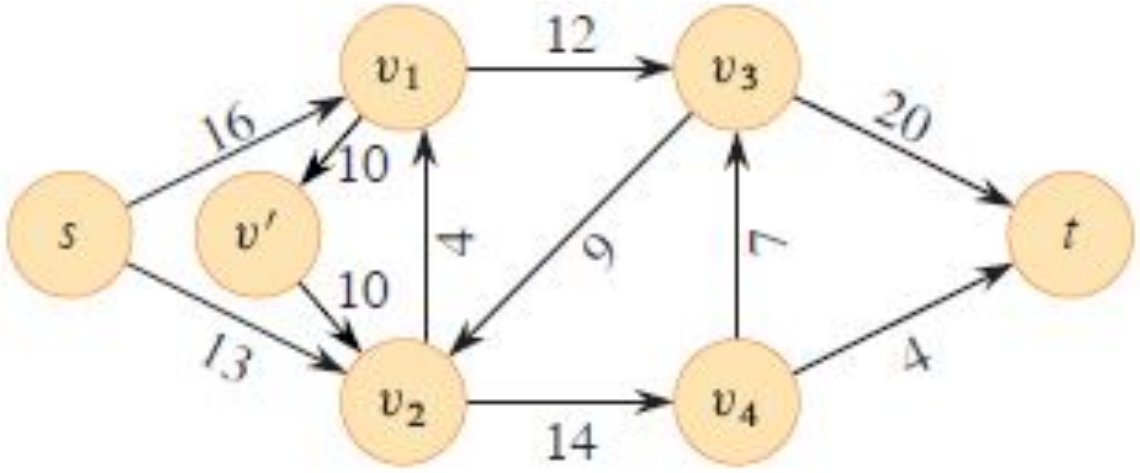
Net flow *f* =23 and **min c(S, T) =23 among all the cut (S, T)**

Flow network Special Cases

- **Multiple edges**
  - **Converting** a **network with antiparallel edges** to an **equivalent one with no antiparallel edges**. **(a)** A flow network containing both the edges (v_1,v_2) and (v_2,v_1). **(b)** An equivalent network with no antiparallel edges. A new vertex v' is added, and one of the antiparallel edges (v_1,v_2) is replaced by the pair of edges (v_1,v') and (v',v_2), both with the same capacity as (v_1,v_2).



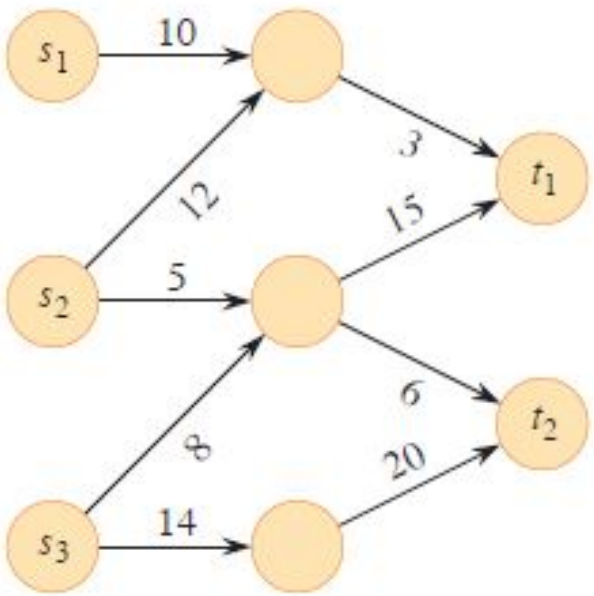
(a)



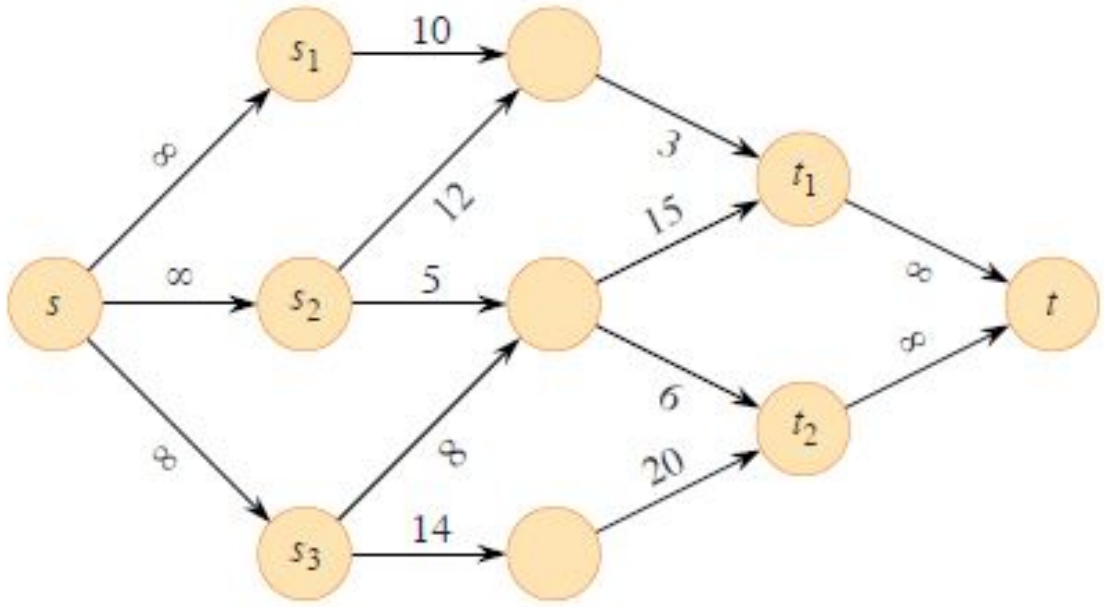
(b)

Flow network Special Cases

- **multiple-source, multiple-sink**
  - **Converting** a **multiple-source, multiple-sink maximum-flow** problem **into a** problem with **a single source and a single sink**. **(a)** A flow network with **three sources** **$S=\{s_1,s_2,s_3\}$** and **two sinks** **$T=\{t_1,t_2\}$**. **(b)** An equivalent single-source, single-sink flow network. **Add a super-source** **s** and **an** **edge** **with** **infinite capacity** **from** **s** **to each of the multiple sources**. Also add a **super-sink** **t** and **an edge with** **infinite capacity** **from each of the multiple sinks to** **t**.



(a)



(b)